

Sales Analytics Dashboard - Project Documentation

1. Overview (Project Kya Hai)

Ye Sales Analytics Dashboard ek Data Analyst portfolio project hai jo Python, Pandas, aur Streamlit ka use karke banaya gaya hai. Isme Node.js ya MERN stack ki tarah koi heavy backend nahi hai. Ye directly cleaned data ko interactive business charts mein convert karta hai.

2. Data Source (Data Kahan Se Aa Raha Hai)

Data 3 cleaned CSV files se aa raha hai jo Backend/Processed/ folder me hain:

- customers_clean.csv
- orders_clean.csv
- products_clean.csv

In files ko pehle hi Jupyter Notebooks aur SQL ka use karke clean kiya gaya tha. App in CSV files ko sidha read karti hai, jisse database query ka wait time bach jata hai aur app fast chalti hai.

3. Data Loading & Caching (Kaise Data Le Raha Hai)

Function: `load_data()`

Ye function sabse important hai. Ye `@st.cache_data` decorator use karta hai. Iska matlab hai ki Streamlit in teeno CSV files ko sirf ek baar memory me load karega. Jab user koi filter change karega, to app wapas hard drive se file read nahi karegi (Caching). Is function ke andar `OrderDate` string ko Python ke actual datetime object mein convert kiya jata hai taaki date-wise filtering ho sake.

4. Data Merging (Data Kaise Jod Raha Hai)

Data load hone ke baad, Pandas ke `.merge()` method (jo SQL ke JOIN ki tarah kaam karta hai) ka use hota hai:

```
merged_df = orders_df.merge(customers_df, on="CustomerID", how="left")
merged_df = merged_df.merge(products_df, on="ProductID", how="left")
```

Ye ek master table (`merged_df`) bana deta hai jisme Order, Customer, aur Product teeno ki details ek saath hoti hain.

5. Interactive Filtering (Filters Kaise Kaam Kar Rahe Hain)

Sidebar mein filters diye gaye hain (Year, Month, Category, City, etc.).

Ye cascading filters hain. Yani, pehle `merged_df` ki copy banti hai (`filtered_df`). Fir line-by-line check hota hai:

- Agar user ne Year = 2025 select kiya, to `filtered_df` sirf 2025 ka data rakhta hai.
- Phir agla filter (jaise Month) bache hue 2025 ke data par hi apply hota hai.

Date Range filter calendar widget (`st.date_input`) se 2 dates leta hai aur data ko un 2 dates ke beech mein filter (`start_date <= Date <= end_date`) karta hai.

6. Helper Functions (Kounsa Function Kya Kaam Kar Raha Hai)

- `load_data()`: CSV files ko disk se read karna aur memory me cache karna.
- `format_currency(value)`: Raw amounts ko Indian readable format me badalna (jaise 200000 ko Rs 2.00 L ya

Sales Analytics Dashboard - Project Documentation

Rs 2.00 Cr).

- `format_number(value)`: Numbers mein commas lagana (jaise 10000 ko 10,000 banana).

7. Dashboard Sections (Kaise Display Ho Raha Hai)

1. KPI Cards: `st.columns()` ka use karke screen ko 7 parts me divide kiya. `filtered_df` par `.sum()` aur `.nunique()` chala kar Total Revenue, Orders, etc. dynamically calculate kiye.
2. Tabs Layout: `st.tabs()` ka use karke poore page ko logical sections (Sales, Products, Customers) me baant diya taki page lamba na ho.
3. Visualizations (Charts):
 - Matplotlib (`plt`) aur Seaborn (`sns`) ka use karke charts draw kiye.
 - `sns.barplot()` se bar charts banaye.
 - `sns.heatmap()` se correlation dikhaya.
 - `ax.pie()` se Order Status aur Payments ke pie charts draw kiye.
 - Har chart draw hone ke baad `st.pyplot(fig)` function us graph ko web page par render (print) kar deta hai.
4. Data Tables: Har graph ke neeche `st.dataframe()` ka use karke exact detailed number wali table show ki gayi hai.

8. Styling & Theming (Light Monochrome Look)

Aapke request ke according, app ko poora Black & White (Light Theme) kar diya gaya hai:

- `.streamlit/config.toml` me background white aur text black set kiya.
- CSS (`st.markdown`) inject karke KPI cards ka background white aur border light-grey kiya.
- Saare Seaborn aur Matplotlib charts ka default color `#333333` (dark grey/black) kar diya aur Heatmap / Pie charts me Greys color palette laga diya.